AiPERF

End-to-End Solution Architecture, Implementation, Deployment and Operations Guide

A reproducible source of proof for standing up the platform from infrastructure to AI-assisted release decisions

Prepared for: Suresh Pittala
Role: Technical Architect / Quality Engineering
Document date: 1 October 2026
Version: 1.0

Purpose This guide captures the target architecture, validated implementation, environment, deployment, execution, evidence flow, governance controls, troubleshooting and production-hardening roadmap needed to reconstruct and operate AiPERF.

Document Control and Use

Field	Value
Document owner	AiPERF solution owner / Technical Architect
Version	1.0
Status	Final implementation reference
Scope	Local/Windows reference deployment with Jenkins, JMeter, InfluxDB OSS 1.8.10, Spring Boot Actuator, Python intelligence services, Streamlit and GPT
Primary database	InfluxDB OSS 1.8.10, database: jmeter
Authoritative codebase	One governed AiPERF Git repository; avoid duplicate Jenkins or legacy script authorities
Security classification	Internal reference; sanitize before external sharing

IP and security boundary Retain or transfer only artifacts that you are authorized to use. Do not copy employer or customer confidential source code, data, documents or credentials into a personal environment. Rotate all exposed API keys and remove secrets from Git history before handover.

How to use this document

1. Use Sections 1 to 4 to understand the product intent and architecture.
2. Use Sections 5 to 9 to prepare the environment and deploy the stack.
3. Use Sections 10 to 14 to execute a run and validate evidence.
4. Use Sections 15 to 18 for operations, security, troubleshooting and production hardening.
5. Use the appendices as command references and acceptance checklists.

Assumptions

- Windows is the primary orchestration host with PowerShell, Git, Python and Jenkins.
- InfluxDB OSS 1.8.10 is used with the jmeter database. This guide intentionally avoids InfluxDB 2.x bucket, organization and token commands.
- JMeter 5.6.3 drives workload through a Gateway service.
- Spring Boot services expose Actuator metrics for Gateway, User, Product and Order services.
- The AI endpoint is enterprise-hosted and configured only through environment variables or Jenkins credentials.
- Repository paths and script names should be adjusted to the final authoritative repository if they differ from this reference.

1. Executive Solution Overview

AiPERF is an AI-native performance engineering intelligence platform. It converts performance execution evidence into governed, explainable release guidance rather than stopping at charts or raw test reports. The core flow is execution data to historical analysis, pattern and anomaly detection, cross-layer correlation, a structured Findings Package, and question-aware GPT guidance through an operator UI.

1.1 Business outcomes

- Detect regressions against an approved and semantically compatible baseline.

- Explain which transactions and microservices most likely contributed to degradation.
- Find comparable historical executions and connect them to known outcomes.
- Separate observed evidence, derived evidence and AI interpretation.
- Issue a governed PROCEED, REVIEW or BLOCK recommendation without suppressing artifacts when a gate fails.
- Provide operators with reproducible evidence through Jenkins, InfluxDB, reports and the AiPERF Copilot UI.

1.2 Validated proof points

Proof point	Validated result
Run correlation	A shared RUN_ID links JMeter, service runtime metrics, comparisons, findings and reports.
Traffic reconciliation	38,732 JMeter samples exactly reconciled to user 12,917 + product 12,910 + order 12,905.
Approved baseline	RUN_286_20260918_014154 represents the clean semantic-V2 median profile.
Degraded run	RUN_290 produced throughput WARNING and P95/P99 CRITICAL anomalies, HIGH risk and BLOCK.
Jenkins governance	A non-proceed release gate completed UNSTABLE while retaining reports and artifacts.
Embeddings	Enterprise embedding contract returned direct 736-float vectors and completed findings generation without warnings.
LLM integration	Primary/failover GPT connectivity and dynamic question answering were validated.

2. Scope, Principles and Non-Goals

2.1 In scope

- Workload execution and JTL evidence capture.
- Run-scoped performance and Spring Boot runtime metric collection.
- Baseline registration and approved-baseline-aware comparison.
- Similar execution retrieval, anomaly detection and correlation intelligence.
- Outcome-aware historical learning and Findings Package assembly.
- AI-assisted release guidance with failover and explicit evidence boundaries.
- Streamlit command-center UI and Grafana operational dashboards.
- Jenkins orchestration, artifact retention and release-gate status handling.

2.2 Architectural principles

Principle	Implementation rule
Evidence first	Every AI conclusion must trace to run-scoped metrics or a clearly marked derived finding.
No silent fallback	If an approved baseline is missing or incompatible, return an explicit status; do not choose another baseline silently.
Semantic compatibility	Compare only histories using compatible workload, feature and telemetry semantics.
Direction awareness	Lower latency/error is better; higher throughput is better; resource metrics require context.

Principle	Implementation rule
Partial evaluation visibility	Missing telemetry must reduce confidence and appear in the UI and findings.
Artifacts survive gate failure	Use Jenkins catchError so BLOCK produces UNSTABLE while reports are still published.
Configuration over hardcoding	Endpoints, credentials, thresholds, ports and baseline IDs come from environment/config.
Single authority	One repository and one Jenkins pipeline are authoritative.

2.3 Non-goals for the reference implementation

- Replacing APM platforms or infrastructure observability.
- Allowing the LLM to calculate authoritative metrics independently.
- Promoting degraded runs as approved baselines.
- Treating missing service telemetry as a pass.
- Using uncontrolled historical runs as compatible evidence.

3. Logical Architecture

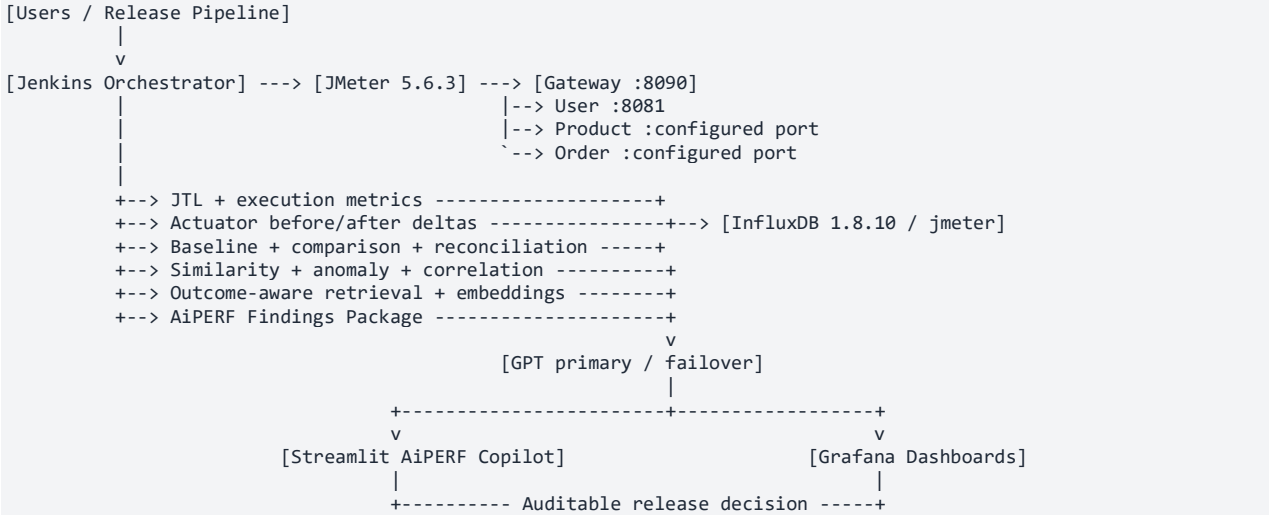


Figure 1. Target logical architecture. The Findings Package is the contract boundary between deterministic intelligence and generative reasoning.

3.1 Layer responsibilities

Layer	Responsibilities	Primary outputs
Execution	Generate controlled workload and raw JTL evidence.	JTL, HTML report, execution metadata
Telemetry	Capture system, JVM, HTTP and service request deltas.	aiperf_service_metrics and run snapshots
Repository	Persist run history, fingerprints, similarities, analysis and outcomes.	InfluxDB measurements

Layer	Responsibilities	Primary outputs
Deterministic intelligence	Compare, reconcile, detect anomalies, correlate and score.	Structured JSON findings
Knowledge	Retrieve semantically similar findings and observed outcomes.	Ranked historical evidence
AI guidance	Answer the operator question using bounded evidence and failover.	Summary, rationale, recommendation, next actions
Experience	Present run health, evidence coverage, findings and conversation.	Streamlit UI, Grafana, Jenkins artifacts

4. Physical Deployment Topology

4.1 Reference single-host topology

Component	Reference host/process	Port / interface
Jenkins	Windows service or java -jar jenkins.war	8080 by default
InfluxDB OSS 1.8.10	influxd.exe Windows process/service	HTTP 8086; backup RPC 8088
JMeter 5.6.3	CLI process launched by Jenkins	Outbound HTTP to Gateway
Gateway	Spring Boot Java process	8090
User service	Spring Boot Java process	8081
Product service	Spring Boot Java process	Project configured
Order service	Spring Boot Java process	Project configured
Streamlit	Python process	8501 by default
Grafana	Windows service/process	3000 by default
GPT/embedding endpoints	Enterprise HTTPS endpoints	443 outbound

4.2 Scale-out production topology

- Run Jenkins agents separately from InfluxDB and Grafana to prevent load-generator contention.
- Deploy JMeter distributed engines only after validating firewall, clock synchronization and identical test assets.
- Place Gateway and microservices on the target-like application environment, not on the load generator for serious benchmarks.
- Use an enterprise secret store such as Jenkins Credentials, Azure Key Vault or HashiCorp Vault.
- Front Streamlit with enterprise SSO/reverse proxy if used beyond a controlled engineering network.

Critical measurement rule Do not interpret tests as production-representative when Jenkins, JMeter, InfluxDB and all

application services compete on the same laptop. The single-host design proves functionality; the scaled topology supports credible capacity conclusions.

5. Technology and Version Baseline

Area	Technology	Reference / constraint
Source control	Git and GitHub	One authoritative AiPERF repository; protected main branch
CI/CD	Jenkins	Pipeline-as-code; credentials binding; catchError for gate
Load generation	Apache JMeter	5.6.3; non-GUI execution
Database	InfluxDB OSS	1.8.10; database jmeter
Visualization	Grafana	Connect using InfluxQL datasource to jmeter
Application	Spring Boot microservices	Actuator enabled and HTTP metrics exposed
Runtime	Python	Use one supported virtual environment and locked requirements
UI	Streamlit	Tabbed AiPERF command center
ML	scikit-learn	StandardScaler, cosine similarity, Isolation Forest; controlled version
AI	Enterprise GPT primary/failover	Question-aware advisor using Findings Package
Embeddings	Enterprise embedding endpoint	736-dimensional vectors validated
Scripting	PowerShell	Windows-friendly setup and operational commands

5.1 Python dependency policy

Create one virtual environment for Jenkins and local execution, freeze exact package versions only after a clean validated run, and make Jenkins invoke that interpreter explicitly. Avoid relying on the interactive user PATH.

```
cd C:\practice\AiPERF
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
pip freeze > requirements-lock.txt
python -c "import streamlit, pandas, sklearn, influxdb; print('Python dependencies OK')"
```

6. Repository and Configuration Standard

6.1 Recommended repository structure

```
AiPERF/
  Jenkinsfile
  README.md
  requirements.txt
  requirements-lock.txt
  .env.example
  .gitignore
  .gitattributes
  config/
    transaction_service_mapping.json
    measurement_contracts.yaml
    thresholds.yaml
  jmeter/
    plans/
    data/
    plugins/
  src/
    collectors/actuator_metrics_collector.py
    repositories/execution_history.py
    repositories/release_outcome_repository.py
    intelligence/similar_execution.py
    intelligence/historical_findings_search.py
    intelligence/anomaly_detection.py
    intelligence/correlation_intelligence.py
    intelligence/bottleneck_intelligence.py
    orchestration/evidence_orchestrator.py
    findings/aiperf_findings_package.py
    ai/embeddings_knowledge_layer.py
    ai/ai_release_advisor.py
    ui/aiperf_ui.py
  tests/
    unit/
    contract/
    integration/
  artifacts/
  docs/
```

6.2 Configuration hierarchy

6. Jenkins credential binding for secrets.
7. Jenkins parameters or environment variables for run-specific settings.
8. Version-controlled non-secret configuration files for mappings and contracts.
9. .env only for local development; never commit it.
10. Code defaults only for safe, non-environment-specific behavior.

6.3 Minimum environment contract

Variable	Purpose	Example / rule
RUN_ID	Global run correlation key	Generated once by Jenkins
INFLUXDB_HOST	InfluxDB host	127.0.0.1
INFLUXDB_PORT	InfluxDB HTTP port	8086
INFLUXDB_DATABASE	Database	jmeter
AIPERF_APPROVED_BASELINE_RUN_ID	Governed comparison target	RUN_286_20260918_014154

Variable	Purpose	Example / rule
AIPERF_FAIL_ON_MISSING_SERVICE_TRAFFIC	Temporary validation behavior	false only until Gateway telemetry handling is finalized
PRIMARY_API_URL / PRIMARY_API_KEY	Primary GPT endpoint and secret	Credential-bound
FAILOVER_API_URL / FAILOVER_API_KEY	Failover endpoint and secret	Credential-bound
EMBEDDING_API_URL / EMBEDDING_API_KEY	Embedding endpoint and secret	Credential-bound
AIPERF_EMBEDDING_DIMENSIONS	Embedding size	736
AIPERF_EMBEDDING_MAX_CHARS	Input guardrail	Set from validated endpoint limits
EMBEDDING_TIMEOUT_SECONDS	Network timeout	Set from operational SLO

Secret handling Store API keys in Jenkins Credentials as Secret Text. Never echo them, place them in .env.example, package them in ZIP files or persist them in build artifacts. Rotate PRIMARY_API_KEY, FAILOVER_API_KEY and EMBEDDING_API_KEY if previously exposed.

7. Infrastructure Setup on Windows

7.1 Prerequisites validation

```
git --version
java -version
py --version
& "C:\Coforge Software\apache-jmeter-5.6.3\bin\jmeter.bat" -v
& "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influxd.exe" version
Get-Service Jenkins -ErrorAction SilentlyContinue
Get-NetTCPConnection -State Listen | Where-Object LocalPort -in 8080,8086,8088,8090,8081,8501,3000
```

7.2 InfluxDB OSS 1.8.10 configuration

Use influxd.exe for server, backup and restore commands. influx.exe is only the query shell. This distinction prevents the common unknown arguments: backup error.

```
# Start server interactively with an explicit config
& "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influxd.exe" `
  -config "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influxdb.conf"

# Open query shell and create the database
& "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influx.exe" `
  -host 127.0.0.1 -port 8086

CREATE DATABASE jmeter
SHOW DATABASES
USE jmeter
SHOW MEASUREMENTS
```

The configuration must use TOML section syntax. Example paths should use forward slashes or escaped backslashes:

```
bind-address = "127.0.0.1:8088"

[meta]
  dir = "C:/influxdb-data/meta"

[data]
  dir = "C:/influxdb-data/data"
  wal-dir = "C:/influxdb-data/wal"

[http]
  enabled = true
  bind-address = ":8086"
```

7.3 Backup and restore

```
# Backup
$Influxd = "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influxd.exe"
$BackupPath = "C:\aiperf_db_backup\jmeter_portable"
New-Item -ItemType Directory -Force -Path $BackupPath | Out-Null
& $Influxd backup -portable -database jmeter -host 127.0.0.1:8088 $BackupPath
if ($LASTEXITCODE -ne 0) { throw "InfluxDB backup failed" }

# Restore into an empty target database
& $Influxd restore -portable -host 127.0.0.1:8088 -db jmeter -newdb jmeter_restored $BackupPath
if ($LASTEXITCODE -ne 0) { throw "InfluxDB restore failed" }
```

7.4 Jenkins service checks

```
Get-Service Jenkins
Start-Service Jenkins
Set-Service Jenkins -StartupType Automatic
Get-NetTCPConnection -LocalPort 8080 -State Listen
Invoke-WebRequest http://localhost:8080/login -UseBasicParsing
```

8. Application Telemetry Setup

8.1 Spring Boot Actuator contract

- Expose health and metrics endpoints on each service.
- Collect system.cpu.usage, process.cpu.usage, jvm.memory.used, jvm.threads.live and http.server.requests.
- Use service-specific base URLs and a shared RUN_ID.
- Capture before and after snapshots, then persist non-negative run deltas for request counts.
- Treat Gateway request telemetry as invalid for transaction comparison when its semantics are known to be corrupted; retain valid Gateway resource evidence.

8.2 Example application properties

```
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
management.metrics.tags.application=${spring.application.name}
management.server.port=${SERVER_PORT}
server.port=${SERVER_PORT}
```

8.3 Endpoint verification

```
$services = @(
  @{Name='gateway'; Url='http://localhost:8090/actuator'},
  @{Name='user'; Url='http://localhost:8081/actuator'}
)
foreach ($s in $services) {
  Invoke-RestMethod "$($s.Url)/health"
  Invoke-RestMethod "$($s.Url)/metrics/system.cpu.usage"
  Invoke-RestMethod "$($s.Url)/metrics/http.server.requests"
}
```

8.4 Transaction-to-service mapping

```
{
  "schema_version": "2",
  "mappings": {
    "GetUser": ["user"],
    "GetProduct": ["product"],
    "CreateOrder": ["order", "product", "user"]
  },
  "unmapped_policy": "report_and_reduce_confidence"
}
```

Mapping rule The mapping file is an evidence contract. A transaction may map to multiple services. Unmapped transactions must be visible, not silently discarded.

9. Jenkins Pipeline Design

9.1 Stage sequence

Stage	Purpose	Required artifact / gate
Initialize	Create RUN_ID, validate paths and load configuration.	run_manifest.json
Environment validation	Check InfluxDB, services, ports, credentials and workload dependencies.	Fail fast if environment is invalid
Pre-run telemetry	Capture Actuator starting counters.	before snapshot
JMeter execution	Run non-GUI workload and generate JTL.	JTL and JMeter HTML report
Post-run telemetry	Capture ending counters and calculate deltas.	service_metrics.json
Ingestion	Write performance and runtime data to InfluxDB.	run-linked measurements
Comparison	Resolve approved baseline and compare compatible metrics.	comparison.json
Reconciliation	Match JMeter traffic to service request deltas.	reconciliation.json
Intelligence	Similarity, anomaly, correlation, bottleneck, outcomes.	structured findings
Findings package	Assemble deterministic evidence contract.	aiperf_findings_package.json
AI advisor	Generate question-aware guidance with failover.	ai_release_advisor_report.json
Release gate	Apply deterministic policy.	PROCEED / REVIEW / BLOCK
Publish	Archive all reports even when gate is non-proceed.	Jenkins artifacts

9.2 Gate behavior pattern

```
stage('Release Gate') {
  steps {
    catchError(buildResult: 'UNSTABLE', stageResult: 'FAILURE') {
      bat '.venv\Scripts\python.exe scripts\release_gate.py --run-id %RUN_ID%'
    }
  }
}
post {
  always {
    archiveArtifacts artifacts: 'artifacts/**', fingerprint: true, allowEmptyArchive: true
    publishHTML(target: [reportDir: 'artifacts/jmeter-report', reportFiles: 'index.html', reportName: 'JMeter
Report', keepAll: true])
  }
}
```

9.3 Pipeline controls

- Never hardcode an approved baseline in Python. Bind AIPERF_APPROVED_BASELINE_RUN_ID from governed configuration.
- Use Jenkins credentials() or withCredentials for all secrets.
- Write artifacts beneath a per-run directory.
- Produce a machine-readable status for missing, incompatible or partial evidence.
- Do not execute the AI stage if the deterministic Findings Package is absent or invalid.

10. Data Model and Measurement Contracts

10.1 Core measurements

Measurement	Purpose	Key tags / fields
aiperf_execution_history	Run-level performance history.	RUN_ID; avg, p90, p95, p99, throughput, errors, samples, duration
aiperf_execution_fingerprint	25-feature normalized execution fingerprint.	RUN_ID; performance, JVM, application and service features
aiperf_service_metrics	Run-scoped service resource and request deltas.	RUN_ID, service; CPU, memory, threads, http_requests
aiperf_similar_execution	Top historical matches.	RUN_ID, similar_run_id; similarity_score, rank
aiperf_analysis	Baseline comparison findings.	RUN_ID, baseline_run_id; deltas, statuses
aiperf_baseline	Baseline registry / metadata.	run_id, approval status, semantic version
release_outcomes	Observed release outcome learning.	RUN_ID; decision, actual outcome, notes

10.2 Field-type discipline for InfluxDB 1.x

InfluxDB field types are fixed within a shard. Persist counts such as request_count_available consistently as integers and ratios consistently as floats. A conflicting write may be rejected or silently create gaps in expected data pipelines.

- Define every measurement, tag and field in a version-controlled contract.
- Validate schema before a Jenkins run.

- Do not reuse a field name with a different type.
- Use RUN_ID as a tag only if cardinality remains operationally acceptable; otherwise use a carefully indexed retrieval strategy and retention policy.

10.3 Example verification queries

```
USE jmeter
SHOW MEASUREMENTS
SHOW FIELD KEYS FROM aiperf_execution_history
SHOW TAG KEYS FROM aiperf_service_metrics
SELECT * FROM aiperf_execution_history WHERE "RUN_ID"='RUN_290' LIMIT 10
SELECT * FROM aiperf_service_metrics WHERE "RUN_ID"='RUN_290' LIMIT 50
```

11. Deterministic Intelligence Pipeline

11.1 Approved-baseline-aware comparison

11. Read AIPERF_APPROVED_BASELINE_RUN_ID.
12. Find the exact baseline and verify approval state.
13. Verify semantic-V2 compatibility, workload identity and required metric availability.
14. If valid, compare only against that baseline.
15. If missing or incompatible, emit APPROVED_BASELINE_UNAVAILABLE or APPROVED_BASELINE_INCOMPATIBLE and do not silently fall back.
16. Calculate direction-aware percentage and absolute deltas.
17. Persist the selected baseline, selection reason and comparison coverage.

11.2 Similar execution intelligence

- Standardize the 25-feature fingerprint using StandardScaler fitted to compatible history.
- Use cosine similarity and return top three matches with 0 to 100 score normalization.
- Use query embeddings during historical findings retrieval, not merely during storage.
- Link similar runs to observed release outcomes and clearly distinguish unknown outcomes.
- Exclude invalid environment runs and degraded runs from baseline candidates.

11.3 Anomaly detection

Metric family	Direction	Illustrative severity logic
Latency: Avg/P90/P95/P99	Lower is better	Positive deviation from compatible history increases severity.
Throughput	Higher is better	Negative deviation increases severity; higher throughput is not an anomaly by default.
Error rate	Lower is better	Any material rise can be critical depending on SLA.
CPU / memory / threads	Context dependent	Evaluate saturation, imbalance and linkage to transaction symptoms.
Request reconciliation	Exact/near exact expected	Unexplained mismatch reduces evidence confidence and may invalidate RCA.

11.4 Correlation and RCA

Correlation Intelligence must traverse transaction symptoms to mapped services to runtime evidence. It should not label a service as root cause merely because it has the highest CPU. A credible finding includes temporal overlap, directional consistency, mapping confidence, metric coverage and alternative explanations.

11.5 Findings Package contract

- Run identity and workload identity.
- Environment validation status.
- Approved baseline status and comparison evidence.
- Transaction and service deltas with reconciliation coverage.
- Anomalies, correlations, bottlenecks and confidence components.
- Similar runs and linked observed outcomes.
- Release governance status and reason codes.
- Evidence limitations and excluded telemetry.
- A compact AI context plus references to detailed artifacts.

12. AI and Embeddings Integration

12.1 Responsibility boundary

Deterministic layer owns	GPT layer may do
Metric calculation, baseline selection, severity, release-gate policy and evidence coverage.	Explain findings, answer user questions, summarize trade-offs and propose investigative next steps.
Source-of-truth run facts and confidence components.	Adapt language for technical, executive or operator audiences.
Missing-data and incompatibility statuses.	State uncertainty without inventing missing facts.

12.2 Question-aware orchestration

- Greeting or general prompt: return a concise conversational response and do not force an executive summary.
- Run-specific question: retrieve the requested run and only necessary evidence.
- Comparison question: include the selected baseline and compatibility status.
- Root-cause question: include mapping, service deltas, temporal support and limitations.
- Release question: include deterministic gate, risk, evidence coverage and outcome history.
- Bound conversation history by turn count or tokens so old evidence cannot contaminate the current run.

12.3 Failover pattern

18. Call the primary model with bounded timeout and validated Findings Package.
19. On retryable network/service failure, invoke the configured failover model.
20. Do not switch models on deterministic validation failures.
21. Record model route, latency, response status and failover reason without logging secrets.
22. If both models fail, show deterministic findings and an explicit AI unavailable status.

12.4 Embedding guardrails

- Validate exactly 736 numeric dimensions before persistence.
- Trim input using AIPERF_EMBEDDING_MAX_CHARS without splitting critical metadata.
- Use EMBEDDING_TIMEOUT_SECONDS and bounded retries.

- Version embedding model, dimensions, normalization and document schema.
- Never compare vectors created by incompatible embedding contracts without an explicit migration.

13. End-to-End Execution Procedure

13.1 Pre-run checklist

- Code checkout is clean and from the authoritative branch/commit.
- Secrets are available through Jenkins Credentials.
- InfluxDB health and jmeter database are verified.
- Gateway, User, Product and Order health endpoints return UP.
- JMeter test data and workload parameters are correct.
- Approved baseline is configured and eligible.
- Clock synchronization is acceptable across hosts.
- Disk space is sufficient for JTL, HTML reports, InfluxDB and Jenkins artifacts.

13.2 Manual smoke execution

```
cd C:\practice\AiPERF
.\.venv\Scripts\Activate.ps1
$env:RUN_ID = "RUN_SMOKE_$(Get-Date -Format yyyyMMdd_HHmmss)"
$env:INFLUXDB_HOST = "127.0.0.1"
$env:INFLUXDB_PORT = "8086"
$env:INFLUXDB_DATABASE = "jmeter"
$env:AIPERF_APPROVED_BASELINE_RUN_ID = "RUN_286_20260918_014154"
python src\collectors\actuator_metrics_collector.py --phase before --run-id $env:RUN_ID
& "C:\Coforge Software\apache-jmeter-5.6.3\bin\jmeter.bat" -n `
  -t jmeter\plans\aiperf_test.jmx `
  -l "artifacts\$env:RUN_ID\results.jtl" `
  -e -o "artifacts\$env:RUN_ID\jmeter-report"
python src\collectors\actuator_metrics_collector.py --phase after --run-id $env:RUN_ID
python src\orchestration\evidence_orchestrator.py --run-id $env:RUN_ID
python src\findings\aiperf_findings_package.py --run-id $env:RUN_ID
python src\ai\ai_release_advisor.py --run-id $env:RUN_ID --question "Is this run safe to release and why?"
```

13.3 Controlled run protocol

23. Warm the application and stabilize caches before measurement.
24. Hold test duration, user model, data set, network path and service topology constant.
25. Collect at least five clean semantic-compatible runs before approving a representative baseline.
26. Use median-based representativeness and reject environment-invalid runs.
27. Execute one intentionally degraded run to validate anomaly, BLOCK and artifact-retention behavior.
28. Never promote the degraded run as a baseline.
29. Record actual release outcome after deployment to close the learning loop.

13.4 Known reference runs

Run	Classification	Use
RUN_285	Invalid environment; microservices off	Exclude from controlled performance history
RUN_284/286/287/288/289	Clean semantic-V2 set	Baseline population
RUN_286_20260918_014154	Representative approved baseline	Governed baseline comparison
RUN_290	Valid degraded run	Anomaly and BLOCK evidence; never baseline

14. Acceptance and Validation

14.1 Functional acceptance

Check	Pass criterion
Run identity	Same RUN_ID appears in JTL-derived data, service metrics, findings and reports.
Traffic reconciliation	JMeter sample count reconciles to downstream service deltas within defined tolerance.
Baseline governance	Exact approved baseline is selected; unavailable/incompatible state has no fallback.
Anomaly direction	Latency increases and throughput decreases are penalized correctly.
Partial evidence	Missing telemetry appears in confidence and UI.
Gate continuity	BLOCK produces UNSTABLE and all artifacts are published.
AI grounding	Response cites Findings Package evidence and does not invent metrics.
Failover	Primary outage routes to failover; double outage leaves deterministic findings visible.
Outcome learning	Observed outcome can be written, retrieved and summarized.
UI	Run selection, health, performance, reconciliation, anomaly, correlation and governance views operate.

14.2 Reference performance baseline

The five clean semantic-V2 runs produced aggregate medians of Avg response time 25.778 ms, P95 54 ms, P99 90 ms, throughput 371.136 RPS, 48,346 samples, 130.265 seconds duration and 0% errors. RUN_286 was selected because it most closely represented those medians.

14.3 Degraded-path proof

RUN_290 recorded throughput 275.177 RPS, P95 87 ms, P99 177 ms and 0% errors. The anomaly engine evaluated all five metrics using five compatible history records, detected throughput WARNING and P95/P99 CRITICAL anomalies, produced HIGH risk and BLOCK, and Jenkins completed UNSTABLE while publishing reports.

15. Streamlit and Grafana Operations

15.1 Start Streamlit with the correct interpreter

```
cd C:\practice\AiPERF
.\.venv\Scripts\Activate.ps1
python -m streamlit run src\ui\aiperf_ui.py --server.address 0.0.0.0 --server.port 8501
```

15.2 UI V2 minimum design

- Global run selector with latest valid run default.
- Environment and dependency health banner.
- Overview tab with run status, risk, gate and evidence coverage.

- Performance tab with latency, throughput, errors and approved-baseline deltas.
- Reconciliation tab with JMeter-to-service counts and mapping coverage.
- Anomaly tab with metric direction, severity and historical context.
- Correlation/RCA tab with transaction-to-service evidence chain.
- Governance tab with baseline status, gate reasons and release outcome.
- Copilot tab with bounded context and visible evidence references.
- Refresh control, cache invalidation and explicit partial-evaluation indicators.

15.3 Grafana restoration

- Add InfluxDB datasource using URL http://127.0.0.1:8086, database jmeter and InfluxQL query language.
- Create run filters using RUN_ID and service tags.
- Separate performance, service runtime, reconciliation, anomaly and release-governance dashboards.
- Do not restore dashboards until backend schema, field types and approved-baseline logic are stable.
- Provision dashboards from version-controlled JSON where possible.

16. Operations, Backup and Handover

16.1 Daily operations

- Verify Jenkins, InfluxDB, application health, disk space and agent connectivity.
- Review the latest run for partial evidence, type conflicts and stale baseline IDs.
- Monitor AI endpoint failures separately from deterministic pipeline health.
- Archive important controlled-run artifacts and retain run manifests.
- Record release outcomes for learning completeness.

16.2 Backup policy

Asset	Method	Minimum recommendation
Git repository	Remote protected repository plus tagged releases	Every validated architecture checkpoint
InfluxDB jmeter	Portable influxd backup	Before cleanup and after approved baseline changes
Jenkins	Back up JENKINS_HOME and credentials securely	Before plugin or pipeline upgrades
Grafana	Export/provision dashboards and datasource config	After accepted dashboard changes
Artifacts	Archive controlled runs and baseline evidence	Keep baseline and degraded proof runs
Secrets	Do not include in backups unless encrypted secret-store backup	Rotate according to policy

16.3 Handover package

- This guide and an architecture diagram.
- Sanitized repository URL, branch policy and tagged commit.
- Environment inventory and port map.
- Non-secret .env.example and Jenkins parameter list.
- Measurement contracts and transaction-service mapping.

- Approved baseline record and evidence supporting its selection.
- Known issues, temporary flags and prioritized backlog.
- Restore test evidence for InfluxDB and Jenkins.
- Named operational owner and credential-rotation confirmation.

17. Security and Governance

17.1 Immediate controls

- Rotate PRIMARY_API_KEY, FAILOVER_API_KEY and EMBEDDING_API_KEY if they appeared in any audit ZIP, log or Git commit.
- Remove tracked __pycache__ directories and .pyc files; expand .gitignore.
- Add .gitattributes and normalize line endings to prevent Windows/Linux drift.
- Scan repository history using an approved secret scanner such as Gitleaks or GitHub secret scanning.
- Restrict Jenkins Script Console and credential access.
- Do not expose InfluxDB 8086, backup RPC 8088, Jenkins 8080 or Streamlit 8501 to untrusted networks.

17.2 Suggested .gitignore

```
.env
.env.*
!.env.example
.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/
.coverage
htmlcov/
artifacts/
*.jtl
jmeter-report/
jenkins_home/
influxdb-data/
*.log
.vscode/settings.json
```

17.3 Audit trail

- Persist code commit, Jenkins build number, RUN_ID, workload checksum and configuration version.
- Record baseline ID, selection result and gate reason codes.
- Record model route and response metadata without prompts containing secrets.
- Preserve deterministic Findings Package as the authoritative decision input.
- Require explicit approval and reason when changing an approved baseline.

18. Troubleshooting Runbook

Symptom	Likely cause	Precise action
unknown arguments: backup	Used influx.exe instead of influxd.exe	Run influxd.exe backup -portable -database jmeter -host 127.0.0.1:8088 <path>.
Influx backup cannot connect	8088 not bound or wrong config	Verify bind-address, restart influxd, then Test-NetConnection 127.0.0.1 -Port 8088.
No module named streamlit.proto	Interpreter/package mismatch or damaged install	Run python -m pip show streamlit, python -m streamlit --version, then reinstall in the exact Jenkins/UI venv.

Symptom	Likely cause	Precise action
Jenkins passes but UI fails	Different environment or interpreter	Launch UI through the same venv and load non-secret config explicitly; do not assume Jenkins env persists.
No service traffic	Services down, mapping wrong or collector scope issue	Check /actuator/health, http.server.requests before/after and mapping file; mark run partial/invalid.
Influx field type conflict	Field written with inconsistent type	Inspect SHOW FIELD KEYS; fix writer contract; use a new measurement/field if historical shard conflicts.
Wrong baseline selected	Fallback logic or hardcoding	Require exact AIPERF_APPROVED_BASELINE_RUN_ID and emit explicit unavailable/incompatible result.
False throughput anomaly	Direction semantics wrong or mixed history	Use lower-is-bad throughput logic and only semantic-compatible history.
AI answer is generic	Evidence orchestrator not question-aware	Retrieve run/question-specific evidence and pass structured Findings Package only.
BLOCK stops publishing	Pipeline exits before post actions	Wrap gate in catchError and archive in post always.

18.1 Diagnostic commands

```
# Ports
Get-NetTCPConnection -State Listen | Sort-Object LocalPort | Format-Table LocalAddress,LocalPort,OwningProcess

# Process behind a port
$pid = (Get-NetTCPConnection -LocalPort 8501 -State Listen).OwningProcess
Get-Process -Id $pid

# Influx health and database
Invoke-WebRequest http://127.0.0.1:8086/ping -UseBasicParsing
& "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influx.exe" -execute "SHOW DATABASES"

# Python interpreter fidelity
Get-Command python
python -c "import sys; print(sys.executable); print(sys.version)"
python -m pip check

# Git hygiene
git status --short
git ls-files | Select-String -Pattern "__pycache__|\.pyc$|\.env$"
```

19. Production-Hardening Roadmap

19.1 P0 before any wider use

30. Rotate all exposed AI and embedding credentials.
31. Remove cached binaries and secrets from tracked history; enforce .gitignore and .gitattributes.
32. Confirm one authoritative Git repository and Jenkinsfile.
33. Finalize Gateway telemetry exclusion so invalid request telemetry cannot contaminate comparison or RCA while valid resource metrics remain available.
34. Back up InfluxDB before cleanup and preserve approved/degraded proof runs.

19.2 P1 before Grafana sign-off

35. Run one final semantic-V2 Jenkins validation with exact approved-baseline enforcement.
36. Validate both normal and degraded approved-baseline paths.

37. Complete direction-aware bottleneck audit.
38. Align readiness score with REFERENCE_ONLY and APPROVED states.
39. Add unit, contract and integration tests for collectors, repositories, baseline selection, anomaly direction, findings schema and AI failover.
40. Validate InfluxDB 1.8.10 field types and standardize on the Python influxdb client.
41. Centralize configuration and measurement contracts.

19.3 P2 intelligence maturation

- Separate availability confidence, evidence quality and model confidence.
- Declare V1/V2 fingerprint compatibility status.
- Consolidate overlapping legacy RCA and summary scripts.
- Mature multivariate and seasonality-aware anomaly detection.
- Add SLA and business-criticality weighting.
- Strengthen outcome-aware historical linkage and feedback quality.
- Restructure Python modules as an installable package with typed interfaces.
- Add workload modelling intelligence, chaos correlation and observability-event enrichment.

19.4 Recommended quality gates

Gate	Target
Unit tests	Critical deterministic modules at least 80% line coverage, with branch tests for direction and missing data.
Contract tests	Every measurement and Findings Package schema validated in CI.
Integration tests	Ephemeral or isolated InfluxDB test database; no production history contamination.
Reconciliation	100% for controlled reference workload or documented tolerance by endpoint semantics.
AI grounding	No numeric claim absent from supplied evidence in sampled evaluation set.
Recovery	Documented restore test for InfluxDB and Jenkins performed at least quarterly.
Security	Zero unrotated exposed secrets and zero committed .env/pyc/cache artifacts.

20. Final Definition of Done

Solution is operational when A new engineer can clone the authoritative repository, provision the documented dependencies, restore or initialize the jmeter database, start services, run Jenkins, produce a correlated RUN_ID, compare against an approved compatible baseline, generate deterministic findings, receive grounded GPT guidance, view results in Streamlit/Grafana, and recover all artifacts after a BLOCK decision.

- Architecture and ownership are documented.
- Exact dependency and configuration baselines are reproducible.
- All services and ports pass health checks.
- One clean and one degraded governed path are validated.
- Approved baseline cannot silently fall back.
- Evidence coverage and partial evaluation are visible.
- Security hardening and credential rotation are complete.

- Backups are restorable.
- Automated tests protect the core contracts.
- A handover owner can execute the runbook without undocumented knowledge.

Appendix A. Quick Start Command Sequence

```
# 1. Clone
cd C:\practice
git clone <authoritative-repository-url> AiPERF
cd AiPERF

# 2. Python
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt

# 3. Start/check InfluxDB
& "C:\Coforge Software\influxdb-1.8.10_windows_amd64\influxdb-1.8.10-1\influxd.exe" -config "<path>\influxdb.conf"
Invoke-WebRequest http://127.0.0.1:8086/ping -UseBasicParsing

# 4. Start application services and verify Actuator
Invoke-RestMethod http://localhost:8090/actuator/health
Invoke-RestMethod http://localhost:8081/actuator/health

# 5. Start Jenkins
Start-Service Jenkins
Start-Process http://localhost:8080

# 6. Run the governed Jenkins job
# Select workload parameters and approved baseline; then Build with Parameters.

# 7. Start UI
python -m streamlit run src\ui\aiperf_ui.py --server.port 8501
```

Appendix B. Handover Acceptance Checklist

☐ Repository URL and branch/tag recorded

☐ Commit hash recorded

☐ Jenkins job exported or pipeline reproducible

☐ Credential IDs documented without values

☐ InfluxDB backup completed and restore tested

☐ Approved baseline status and supporting evidence retained

☐ RUN_290 degraded proof retained but not baseline eligible

☐ Transaction-service mapping validated

☐ Measurement contracts versioned

☐ Streamlit starts with intended interpreter

☐ Grafana datasource and dashboards exported

☐ Known issues and temporary flags documented

☐ All exposed keys rotated

☐ No confidential artifacts transferred without authorization

☐ New owner completes one supervised run

Appendix C. Decision Record Summary

Decision	Reason

Decision	Reason
InfluxDB OSS 1.8.10 / jmeter remains the operational store	Matches the validated environment and existing measurement history.
Approved baseline is explicit and governed	Prevents accidental comparison against degraded or incompatible history.
RUN_286 is the representative reference	Closest match to medians of five clean semantic-V2 runs.
RUN_290 retained only as degraded evidence	Proves anomaly and gate behavior but is not baseline eligible.
Findings Package precedes GPT	Preserves deterministic evidence and auditability.
Streamlit remains UI technology for V2	Fastest path from prototype to tabbed engineering command center.
Grafana follows backend hardening	Avoids rebuilding dashboards against unstable schema.
Jenkins BLOCK maps to UNSTABLE with artifacts	Communicates risk without destroying diagnostic evidence.

Appendix D. Known Temporary Conditions

- AIPERF_FAIL_ON_MISSING_SERVICE_TRAFFIC=false is temporary until Gateway telemetry handling is finalized.
- Old external workload history must be isolated from the local controlled workload before broad historical analysis.
- Gateway request telemetry may be excluded from comparison/RCA while resource-only evidence is retained.
- Product and Order ports must be taken from the authoritative service configuration.
- Thresholds for embedding input length, timeout and operational SLO must be finalized from the enterprise endpoint contract.